

SmartSplit

SmartSplit est une application web full-stack responsive conçue pour faciliter la gestion et le partage des dépenses au sein d'un groupe (colocation, voyage, événement ou toute autre activité commune). Développée avec **React, Node.js, Express et MySQL**, elle automatise le calcul des soldes entre les membres et propose des règlements permettant d'équilibrer les comptes tout en réduisant le nombre de transactions. L'application repose sur une architecture en plusieurs couches et une API REST sécurisée.

1. ARCHITECTURE & PATRONS DE CONCEPTION

Architecture en couches (Service – Repository)

Le Back-end est organisé en plusieurs couches distinctes (Routes, Controllers, Services et Repositories) afin de séparer les responsabilités. Cette organisation facilite la maintenance, l'évolution et la réutilisation du code.

Architecture Front-end modulaire

L'interface utilisateur est développée avec React selon une approche orientée composants. La gestion de l'état s'appuie sur le Context API, tandis que les échanges avec l'API REST sont centralisés dans une couche dédiée.

2. FONCTIONNALITÉS PRINCIPALES

Le serveur expose une API REST permettant de gérer l'ensemble des fonctionnalités de l'application :

- **Gestion des foyers (Households)** : création, modification et suppression des foyers ainsi que gestion des membres.
- **Gestion des dépenses** : ajout, modification et suppression des dépenses avec prise en charge des différents modes de répartition.
- **Calcul des soldes** : mise à jour automatique des soldes de chaque membre après chaque dépense ou remboursement.
- **Règlements (Settlements)** : génération des remboursements nécessaires afin d'équilibrer les comptes en limitant le nombre de transactions.
- **Historique des activités** : suivi chronologique des principales opérations réalisées au sein d'un foyer.
- **Export des données** : génération d'exports permettant de conserver un historique des dépenses et des règlements.

3. SÉCURITÉ, INFRASTRUCTURE ET VALIDATION

- **Authentification JWT** : sécurisation des routes grâce à des JSON Web Tokens.
- **Protection des mots de passe** : stockage sécurisé des mots de passe à l'aide de l'algorithme bcrypt.
- **Validation des données** : vérification des données reçues avant leur traitement afin de garantir leur cohérence.
- **Sécurisation de l'API** : utilisation de middlewares tels que Helmet, CORS et gestion centralisée des erreurs.
- **Configuration sécurisée** : stockage des informations sensibles dans des variables d'environnement (.env).

Stack technique

Front-end

- React
- React Router
- HTML5
- Tailwind CSS
- Context API
- TanStack Query
- Axios

Back-end

- Node.js
- Express.js
- API REST
- JWT
- bcrypt
- Helmet
- CORS

Base de données

- MySQL
- Requêtes SQL paramétrées

Outils

- Git
- npm
- ESLint
- Dotenv